

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Лабораторная работа №1

по дисциплине «Проектирование защищенных интеллектуальных
информационных систем»

на тему «Управление доступом с помощью списков контроля доступа.
Часть 2. Управление доступом к приложениям и системам баз данных»

Выполнил студент гр. 321701

Ковальчук В. Н.
Самович В. М.

Проверил

Сальников Д. А.

Минск 2026

Цель: изучить механизмы контроля доступа, предоставляемые серверными приложениями и системами управления базами данных, и получить практические навыки настройки политики безопасности (администратор – пользователь – гость) для СУБД и веб-приложений.

Задание:

- а) Создать программу, которая позволяет установить веб-приложение или систему управления базами данных (СУБД).
- б) Создать программу, которая позволяет настроить механизм контроля доступа для выбранного веб-приложения или СУБД.
- в) Создать программу для проверки корректности работы настроенной политики безопасности выбранной СУБД или веб-приложения.
- г) Создать программу, которая позволяет удалить веб-приложение или СУБД, а также созданные ими объекты файловой системы.
- д) Оформить отчёт о проделанной работе.

Политика безопасности должна быть настроена следующим образом: в системе должны присутствовать администратор системы с полным доступом ко всем подсистемам и максимальными привилегиями, пользователь с полными либо частичными правами в подсистеме, предоставленной администратором, и гость с правами только на чтение отдельных фрагментов, предоставленных администратором.

Задание выполнено для двух вариантов вместо одного: документоориентированной СУБД MongoDB и веб-приложения обмена сообщениями OpenFire. Для каждого варианта реализован отдельный консольный C#-проект (Lab1.2-MongoDB и Lab1.2-OpenFire), последовательно выполняющий все четыре программы из задания как этапы одного сценария: установка, настройка контроля доступа, проверка и удаление. Все действия – реальные команды Linux (`docker`, `apt-get`, `dpkg`, `mongosh`, `curl` и т. п.), запускаемые как внешние процессы; .NET здесь выступает только оркестратором сценария, а не исполнителем самой политики безопасности.

Краткое описание MongoDB

MongoDB – документоориентированная СУБД: данные хранятся не в таблицах со строками, а в коллекциях JSON-подобных документов (BSON). В лабораторной работе используется официальный Docker-образ `mongo:7`. По умолчанию свежий экземпляр MongoDB принимает любые подключения без аутентификации – контроль доступа необходимо включить явно ключом `-auth` и настроить отдельно от установки. Модель прав в MongoDB строится вокруг ролей: встроенные роли (например, `root`, `readWrite`) назначаются пользователю в контексте конкретной базы данных, а для более точных

ограничений можно определить собственную роль с перечнем разрешённых действий над конкретной коллекцией – этим MongoDB напоминает списки контроля доступа из первой части лабораторной работы, только уровнем выше: не файл и не каталог, а документ и коллекция.

Краткое описание OpenFire

OpenFire – сервер обмена мгновенными сообщениями по протоколу XMPP (Jabber), написанный на Java. Он состоит из двух независимых подсистем контроля доступа: во-первых, веб-консоль администрирования (порт 9090), доступ к которой разрешён только учётным записям из списка администраторов; во-вторых, сам протокол XMPP, в котором права участников многопользовательских чат-комнат (Multi-User Chat, MUC) определяются их *аффилиацией* (владелец, администратор, участник, не участвует) и производной от неё *ролью* в конкретной комнате (модератор, участник, посетитель). В комнате с включённой модерацией только участники с ролью не ниже «участник» имеют право голоса – посетители (в том числе анонимные) могут только читать переписку.

Реализация: MongoDB

Проект Lab1.2-MongoDB организован так же, как и программа из первой части лабораторной работы: `Program.cs` последовательно вызывает четыре этапа из каталога `Steps` (`Install`, `Configure`, `Verify`, `Cleanup`), а вспомогательный класс `Report` запускает внешние процессы и печатает результат каждой команды в формате `[OK]/[FAIL]`, а результат проверки прав доступа – в формате `[ALLOWED]/[DENIED]`.

1. Установка

Установка выполняется командами `docker pull mongo:7` и `docker run`, с монтированием именованного тома `mongo_lab_data` для постоянного хранения данных. Контейнер намеренно запускается без `-auth`: MongoDB разрешает создать самого первого пользователя без пароля только через т. н. `localhost exception`, то есть до включения обязательной аутентификации.

2. Настройка контроля доступа

Пока аутентификация ещё не включена, программа создаёт администратора `admin` с встроенной ролью `root`, после чего контейнер перезапускается с ключом `-auth` (том с данными переиспользуется, поэтому созданный пользователь сохраняется). Далее от имени администратора создаются: пользователь `labuser` с ролью `readWrite` на базе `labdb` (полные права в предоставленной администратором «подсистеме» – одной прикладной базе данных) и пользователь `labguest`, которому назначена не встроенная, а специально созданная роль `guestReader`, разрешающая только действие `find` и только над одной коллекцией `public_notes` (см. листинг 1). Соседняя коллекция `private_notes` для гостя недоступна вовсе.

```
1 Report.Step(8, $"Create '{Config.GuestUser}' - read-only on
   ↳ '{Config.AppDb}.{Config.PublicCollection}' only");
2 MongoEvalAuth($"Create role '{Config.GuestRole}' (find on
   ↳ {Config.AppDb}.{Config.PublicCollection} only)",
3   $"db.getSiblingDB('{Config.AppDb}').createRole({{role: '{Config.GuestRol }
   ↳ e}', "
   ↳ +
4   $"privileges: [{resource: {{db: '{Config.AppDb}', collection: '{Config.Publ }
   ↳ icCollection}' }}, actions: ['find'] } }], roles: [ ] } }");
5 MongoEvalAuth($"Create user '{Config.GuestUser}' (role {Config.GuestRole})",
6   $"db.getSiblingDB('{Config.AppDb}').createUser({{user: '{Config.GuestUse }
   ↳ r}', pwd: '{Config.GuestPassword}', roles: [{role: '{Config.GuestRole}' }
   ↳ , db: '{Config.AppDb}' } } } }");
```

Листинг 1 – Создание гостевой роли и пользователя с правом чтения одной коллекции

3. Проверка корректности

Проверка выполняется той же утилитой `mongosh`, но уже от имени каждой из трёх учётных записей, и интерпретирует нулевой код завершения команды как `ALLOWED`, а ненулевой (сервер отклонил операцию как неавторизованную) – как `DENIED`. Полный протокол реального запуска программы приведён ниже (см. листинг 2). Администратор проходит любую проверку; `labuser` может читать и писать в `labdb`, но не может ни создать нового пользователя, ни обратиться к посторонней базе `otherdb`; `labguest` может прочитать только `public_notes` – чтение `private_notes` и любая запись отклоняются. Отдельно проверено, что клиент без учётных данных не может выполнить вообще ни одной операции – `-auth` действительно принудительно применяется сервером, а не является лишь опцией по умолчанию.

```
1 == Step 1: Pull the MongoDB server image ==
2 [OK] Pull MongoDB image $ docker pull mongo:7
3 == Step 2: Create a persistent data volume ==
4 [OK] Create data volume $ docker volume create mongo_lab_data
5 == Step 3: Start MongoDB (bootstrap mode, no authentication yet) ==
6 [OK] Run MongoDB container $ docker run -d --name mongo_lab -p 27017:27017 -v mongo_lab_data:/data/db mongo:7
7 [OK] MongoDB accepted a connection and is ready
8 == Step 4: Create the administrator account through the localhost exception ==
9 [OK] Create admin user (role 'root' on 'admin') $ docker exec mongo_lab mongosh --quiet --eval
  ↪ db.getSiblingDB('admin').createUser({user:'admin',pwd:'AdminP@ss1',roles:[{role:'root',db:'admin'}]})
10 == Step 5: Restart MongoDB with authorization enforced ==
11 [OK] Stop the bootstrap container $ docker rm -f mongo_lab
12 [OK] Run MongoDB container with --auth $ docker run -d --name mongo_lab -p 27017:27017 -v
  ↪ mongo_lab_data:/data/db mongo:7 --auth
13 [OK] MongoDB is back up with authorization enforced
14 == Step 6: Seed the application database with public and private data ==
15 [OK] Insert a document into public_notes
16 [OK] Insert a document into private_notes
17 == Step 7: Create 'labuser' - readWrite on 'labdb' only ==
18 [OK] Create user 'labuser' (role readWrite on labdb)
19 == Step 8: Create 'labguest' - read-only on 'labdb.public_notes' only ==
20 [OK] Create role 'guestReader' (find on labdb.public_notes only)
21 [OK] Create user 'labguest' (role guestReader)
22 == Step 9: Verify the administrator's privileges ==
23 [ALLOWED] admin ListDatabases admin (cluster-wide)
24 == Step 10: Verify 'labuser' has full rights inside 'labdb' only ==
25 [ALLOWED] labuser Read labdb.public_notes
26 [ALLOWED] labuser Write labdb.private_notes
27 [DENIED ] labuser CreateUser labdb (admin-only op)
28 [DENIED ] labuser Read otherdb.secrets (no role granted)
29 == Step 11: Verify 'labguest' can only read 'public_notes' ==
30 [ALLOWED] labguest Read labdb.public_notes
31 [DENIED ] labguest Read labdb.private_notes
32 [DENIED ] labguest Write labdb.public_notes
33 == Step 12: Verify that an unauthenticated client is rejected outright ==
34 [DENIED ] (none) Write labdb.public_notes (no credentials)
35 == Step 13: Remove MongoDB, its data volume and its image ==
36 [OK] Remove the container $ docker rm -f mongo_lab
37 [OK] Remove the data volume $ docker volume rm mongo_lab_data
38 [OK] Remove the MongoDB image $ docker rmi mongo:7
39 === Summary ===
40 Operations: 16 total, 0 failed
41 CreateUser allowed 0 / 1 denied 1
42 ListDatabases allowed 1 / 1 denied 0
43 Read allowed 2 / 4 denied 2
44 Write allowed 1 / 3 denied 2
```

Листинг 2 – Реальный протокол работы программы Lab1.2-MongoDB

4. Удаление

Завершающий этап удаляет контейнер, именованный том с данными и сам образ `mongo:7` – после него от установки MongoDB не остаётся ни одного файла и ни одного объекта Docker.

Реализация: OpenFire

Проект `Lab1.2-OpenFire` устроен по той же схеме (`Install` → `Configure` → `Verify` → `Cleanup`), но, в отличие от MongoDB, официального Docker-образа у OpenFire нет, поэтому установка выполняется «с нуля» – пакетом `.deb` внутри обычного контейнера `ubuntu:24.04` – в точности так, как это делал бы администратор на настоящей виртуальной машине.

1. Установка

В контейнер устанавливается пакет `default-jre-headless` (Java, необходимая OpenFire), после чего с официального сайта `download.igniterealtime.org` загружается пакет `openfire_5.1.2_all.deb` и устанавливается командой `dpkg -i`. Туда же устанавливается виртуальное окружение Python с библиотекой `slixmpp` – она используется не для настройки OpenFire, а исключительно для последующей проверки прав доступа непосредственно по протоколу XMPP (см. пункт 3).

2. Настройка контроля доступа

OpenFire обычно настраивается через мастер первого запуска в веб-интерфейсе, что не поддаётся автоматизации без эмуляции браузера. Вместо этого используется официально документированный, но малоизвестный механизм `<autosetup>` в конфигурационном файле `openfire.xml`: если в нём присутствует блок `autosetup` с `run=true`, сервер проходит мастер настройки автоматически при первом запуске и удаляет этот блок из файла. Таким способом заранее задаются пароль администратора `admin` и создаётся обычная учётная запись `iuser`, а в конфигурацию явно прописывается анонимная XMPP-аутентификация – она нужна учётной записи «гость», у которой нет и не должно быть пароля.

После запуска сервера программа подключается к нему уже не через HTTP, а непосредственно по протоколу XMPP (библиотека `slixmpp`, роль администратора): создаёт комнату `labroom`, включает в её настройках модерацию (`moderatedroom`) и явно назначает `iuser` аффилиацию `member` – то есть именно администратор предоставляет пользователю право голоса в этой комнате (см. листинг 3). Учётная запись «гость» отдельно не создаётся вовсе: анонимный участник входит в ту же комнату без пароля и получает роль «посетитель», которая в модулируемой комнате не даёт права говорить – только читать.

```

1 form = await self.plugin["xep_0045"].get_room_config(ROOM)
2 form.set_values({
3     "muc#roomconfig_roomname": "Lab Room",
4     "muc#roomconfig_moderatedroom": True,
5     "muc#roomconfig_publicroom": True,
6     "muc#roomconfig_persistentroom": True,
7     "muc#roomconfig_memberonly": False,
8     "muc#roomconfig_whois": "moderators",
9 })
10 await self.plugin["xep_0045"].set_room_config(ROOM, form)
11
12 # Admin grants the "user" identity full (voice) rights in this subsystem.
13 await self.plugin["xep_0045"].set_affiliation(ROOM, affiliation="member", jid=USER_JID)

```

Листинг 3 – Настройка модулируемой комнаты и назначение пользователю права голоса

3. Проверка корректности

Проверка охватывает обе подсистемы контроля доступа OpenFire независимо друг от друга. Доступ к веб-консоли администрирования проверяется реальным HTTP-логингом через `curl` (с извлечением CSRF-токена из формы и последующей отправкой формы входа): `admin` успешно получает сессию и перенаправление на `index.jsp`, а `iuser` с верным паролем получает страницу с ошибкой «Login failed... make sure... you're an admin» – OpenFire различает «валидная учётная запись» и «учётная запись администратора» на уровне самой веб-консоли. Доступ к чат-комнате проверяется на уровне протокола: небольшой XMPP-клиент на Python подключается под нужной личностью, входит в комнату `labroom` и пытается отправить сообщение, а затем проверяет, не вернул ли сервер ответной ошибки `forbidden`. Полный протокол реального запуска приведён ниже (см. листинг 4): и `admin`, и `iuser` могут говорить в комнате, тогда как анонимный гость и подключение с несуществующей учётной записью получают отказ.

```

1 == Step 1: Start a bare Linux container to install Openfire into ==
2 [OK] Run base container $ docker run -d --name openfire_lab -p 9090:9090 -p 9091:9091 -p
   ↪ 5222:5222 -p 5269:5269 ubuntu:24.04 sleep infinity
3 == Step 2: Install the Java runtime Openfire needs ==
4 [OK] Refresh package index
5 [OK] Install default-jre-headless, wget, python3-venv
6 == Step 3: Download and install the official Openfire package ==
7 [OK] Download the Openfire .deb package $ wget openfire_5.1.2_all.deb
8 [OK] Install the Openfire package $ dpkg -i /tmp/openfire.deb
9 == Step 4: Install a small Python XMPP client for the later protocol-level checks ==
10 [OK] Create a virtualenv for test tooling
11 [OK] Install slixmpp into the virtualenv
12 == Step 5: Provision Openfire unattended (autosetup) with the admin and user accounts ==
13 [OK] Copy the autosetup configuration into the container
14 [OK] Fix ownership of the configuration file
15 [OK] Fix permissions of the configuration file
16 == Step 6: Start Openfire and wait for the autosetup run to complete ==
17 [OK] Start the Openfire service $ /etc/init.d/openfire start
18 [OK] Admin Console is up and autosetup has completed
19 == Step 7: Create the 'labroom' room and grant 'iuser' member rights in it ==
20 [OK] Copy the room setup script into the container
21 [OK] Configure the chat room's access control
22 == Step 8: Verify who can log into the Admin Console ==
23 [ALLOWED] admin AdminConsoleLogin /index.jsp
24 [DENIED ] iuser AdminConsoleLogin /index.jsp
25 [DENIED ] admin (wrong password) AdminConsoleLogin /index.jsp
26 == Step 9: Verify who can speak in 'labroom' (moderated: only members and the owner get
   ↪ voice) ==
27 [OK] Copy the room probe script into the container
28 [ALLOWED] admin SpeakInRoom labroom@conference.localhost
29 [ALLOWED] iuser SpeakInRoom labroom@conference.localhost
30 [DENIED ] guest (anonymous) SpeakInRoom labroom@conference.localhost
31 [DENIED ] attacker (unknown account) SpeakInRoom labroom@conference.localhost
32 == Step 10: Remove Openfire and every file it created ==
33 [OK] Remove the container $ docker rm -f openfire_lab
34 === Summary ===
35 Operations: 16 total, 0 failed
36 AdminConsoleLogin allowed 1 / 3 denied 2
37 SpeakInRoom allowed 2 / 4 denied 2

```

Листинг 4 – Реальный протокол работы программы Lab1.2-OpenFire

4. Удаление

Поскольку OpenFire был установлен не в отдельный именованный том, а непосредственно в файловую систему контейнера, для полного удаления достаточно одной команды `docker rm -f` – вместе с контейнером удаляются и сам сервер, и встроенная база данных, и созданные плагины, без необходимости отдельно подчищать файлы на диске.

Выводы

В ходе выполнения лабораторной работы был освоен механизм контроля доступа на уровне приложений и систем управления базами данных – в отличие от первой части, где права ограничивались правами файловой системы, здесь для каждой из двух рассмотренных систем (MongoDB, OpenFire) реализована и экспериментально подтверждена трёхуровневая политика безопасности «администратор – пользователь – гость», при этом для каждой системы это разграничение выражается собственными, специфичными для неё средствами: у MongoDB это встроенные и пользовательские роли с привилегиями на уровне базы данных и коллекции, у OpenFire – отдельно список администраторов веб-консоли и отдельно аффилиации и роли участников чат-комнаты по протоколу XMPP. Практически показано, что, несмотря на разную природу этих механизмов, в основе обоих лежит один и тот же принцип наименьших привилегий: пользователю и гостю явно и по отдельности выдаётся ровно тот объём прав, который необходим для их роли, а не урезается из более широкого набора.

Семантически лабораторная работа подтвердила, что задача администратора не сводится к однократной установке ПО: то, что MongoDB по умолчанию разрешает подключение вовсе без аутентификации, а OpenFire по умолчанию не различает пользователей и администраторов до отдельной настройки, означает, что первым шагом всегда является явное включение и настройка контроля доступа – то же наблюдение, что и в первой части работы, только теперь на уровне не операционной системы, а конкретного приложения. С прагматической точки зрения работа показала, что настройка безопасности не всегда сводится к формам в веб-интерфейсе: там, где графический мастер настройки не поддается безопасной автоматизации (как у OpenFire), у хорошо спроектированного приложения, как правило, находится задокументированный неинтерактивный путь (файл `autosetup` в `openfire.xml`), а там, где такого пути нет, сам протокол приложения (XMPP) оказывается более удобным и надёжным инструментом автоматизации, чем эмуляция браузера. Использование Docker-контейнеров для установки обеих систем также оказалось практичным решением: оно позволило безопасно выполнять реальные, а не имитационные команды установки и настройки, и гарантировало полное и воспроизводимое удаление всех созданных объектов на завершающем этапе.